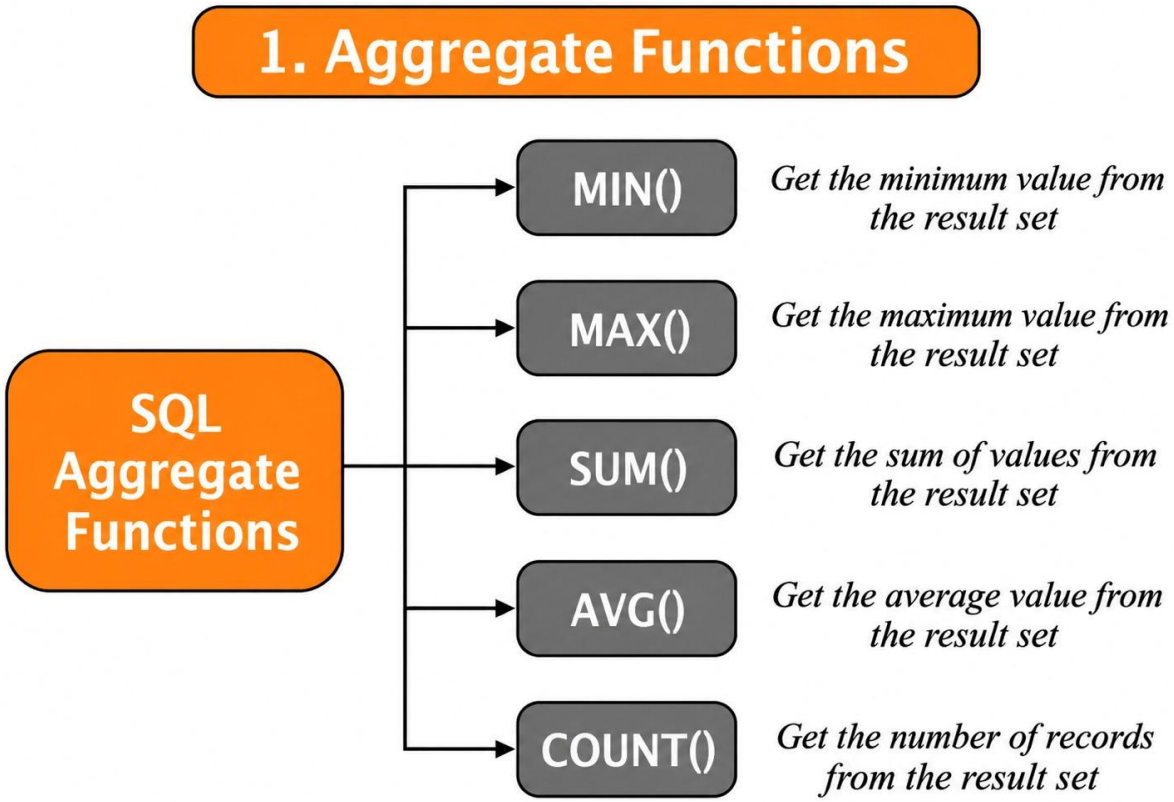
SALES AND CUSTOMER ANALYTICS SYSTEM

Week: 9

1. Aggregate Functions

Definition

Aggregate functions are SQL functions used to perform calculations on a group of rows and return a single result.



1.1 COUNT()

Find the total number of orders:

```
SQL> SELECT COUNT(*) AS Total_Customers
  2  FROM Customer;

TOTAL_CUSTOMERS
---------------
              3
```

1.2 SUM()

Find the total sales amount:

```
SQL> SELECT SUM(Total_Amount) AS Total_Sales
  2  FROM Orders;

TOTAL_SALES
-----------
     195997
```

1.3 AVG()

Find the average order amount:

```
SQL> SELECT AVG(Price) AS Average_Price
  2  FROM Product;

AVERAGE_PRICE
-------------
        54999
```

1.4 MIN()

Find the lowest order amount:

```
SQL> SELECT MIN(Price) AS Minimum_Price
  2  FROM Product;

MINIMUM_PRICE
-------------
        24999
```

1.5 MAX()

Find the highest order amount:

```
SQL> SELECT MAX(Price) AS Maximum_Price
  2  FROM Product;

MAXIMUM_PRICE
-------------
        74999
```

2. Generate Total Sales Reports

A sales report can be generated using the SUM() function on the Total_Amount column.

```
SQL> SELECT SUM(Total_Amount) AS Total_Sales
  2  FROM Orders;

TOTAL_SALES
-----------
     195997
```

2.1 Customer-wise Sales Report

To find how much each customer has purchased:

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name,
  3         C.Last_Name,
  4         SUM(O.Total_Amount) AS Total_Sales
  5  FROM Customer C
  6  JOIN Orders O
  7  ON C.Customer_ID = O.Customer_ID
  8  GROUP BY C.Customer_ID, C.First_Name, C.Last_Name
  9  ORDER BY Total_Sales DESC;

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          TOTAL_SALES
-------------------------------------------------- -----------
        201 Arun
Kumar                                                    84999

        202 Priya
Sharma                                                   64999

        203 Rahul
Raj                                                      45999
```

Purpose This report shows:

- Customer ID
- Customer name
- Total purchase amount
- Customers arranged according to purchase amount

3. Find Top Customers Based on Purchase Amount

The SUM() function can be combined with GROUP BY and ORDER BY to identify customers with higher purchase amounts.

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name,
  3         C.Last_Name,
  4         SUM(O.Total_Amount) AS Total_Purchase_Amount
  5  FROM Customer C
  6  JOIN Orders O
  7  ON C.Customer_ID = O.Customer_ID
  8  GROUP BY C.Customer_ID, C.First_Name, C.Last_Name
  9  ORDER BY Total_Purchase_Amount DESC;

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          TOTAL_PURCHASE_AMOUNT
-------------------------------------------------- ---------------------
        201 Arun
Kumar                                                              84999

        202 Priya
Sharma                                                             64999

        203 Rahul
Raj                                                                45999
```

4. Identify Best-Selling Product

The Order_Details table contains the quantity of each product ordered.
Therefore, we can use SUM(Quantity) to find the total quantity sold for each product.

```
SQL> SELECT P.Product_ID,
  2         P.Product_Name,
  3         SUM(OD.Quantity) AS Total_Quantity_Sold
  4  FROM Product P
  5  JOIN Order_Details OD
  6  ON P.Product_ID = OD.Product_ID
  7  GROUP BY P.Product_ID, P.Product_Name
  8  ORDER BY Total_Quantity_Sold DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
       101
iPhone 15
                  2

       102
Samsung Galaxy S24
                  1

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
```

Purpose This query:

1. Connects Product and Order_Details.

2. Groups records according to product.

3. Calculates total quantity sold.

4. Displays products from highest to lowest quantity sold.

4.1 Product Sales Amount

Instead of only calculating quantity, we can calculate the total sales generated by each product.

```
SQL> SELECT P.Product_ID,
  2         P.Product_Name,
  3         SUM(OD.Quantity * OD.Unit_Price) AS Total_Sales_Amount
  4  FROM Product P
  5  JOIN Order_Details OD
  6  ON P.Product_ID = OD.Product_ID
  7  GROUP BY P.Product_ID, P.Product_Name
  8  ORDER BY Total_Sales_Amount DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_SALES_AMOUNT
------------------
       101
iPhone 15
            159998

       102
Samsung Galaxy S24
             64999

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_SALES_AMOUNT
------------------
```

5. Category-wise Sales Analysis

Category-wise analysis helps determine the sales generated by each product category.

```
SQL> SELECT C.Category_ID,
  2         C.Category_Name,
  3         SUM(OD.Quantity) AS Total_Quantity_Sold,
  4         SUM(OD.Subtotal) AS Total_Sales_Amount
  5  FROM Category C
  6  JOIN Product P
  7  ON C.Category_ID = P.Category_ID
  8  JOIN Order_Details OD
  9  ON P.Product_ID = OD.Product_ID
 10  GROUP BY C.Category_ID, C.Category_Name
 11  ORDER BY Total_Sales_Amount DESC;

CATEGORY_ID
-----------
CATEGORY_NAME
--------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_SALES_AMOUNT
------------------- ------------------
          1
Mobiles
                  3             224997
```

6. Combined Business Sales Report

A complete sales report can combine customer, order, product, and category information.

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name,
  3         C.Last_Name,
  4         O.Order_ID,
  5         P.Product_Name,
  6         CAT.Category_Name,
  7         OD.Quantity,
  8         OD.Unit_Price,
  9         OD.Subtotal,
 10         PY.Payment_Method,
 11         PY.Payment_Status
 12  FROM Customer C
 13  JOIN Orders O
 14  ON C.Customer_ID = O.Customer_ID
 15  JOIN Order_Details OD
 16  ON O.Order_ID = OD.Order_ID
 17  JOIN Product P
 18  ON OD.Product_ID = P.Product_ID
 19  JOIN Category CAT
 20  ON P.Category_ID = CAT.Category_ID
 21  JOIN Payment PY
 22  ON O.Order_ID = PY.Order_ID
 23  ORDER BY O.Order_ID;

no rows selected
```

Report Structure

This report can be used to understand:

- Who purchased the product
- Which order was placed
- Which product was purchased
- Which category the product belongs to
- Quantity purchased
- Amount generated